10장 배포 관리와 태그

이 장에서는 완성된 코드의 관리와 배포 방법을 알아보겠습니다. 깃은 코드를 안정된 상태로 유지하며 완성된 코드를 사용자에게 배포합니다.

10.1 배포

10.2 버전

10.3 태그

10.4 태그 목록

10.5 Annotated 태그

10.6 Lightweight 태그

10.7 특정 커밋 태그

10.8 태그를 사용한 체크아웃

10.9 태그 공유

10.10 정리

10.1 배포

프로그램을 개발한 후에는 완성된 결과물을 최종 사용자에게 전달해야 합니다. 이때 최종 사용자에게 전달하는 과정을 배포라고 합니다.

개발을 완료했다고 해서 결과물을 직접 사용자에게 배포할 수 없습니다. 완성된 형태로 결과물을 배포하려면 코드를 정리하는 작업들이 추가로 필요합니다. 개발 단계에서 발생한 테스트 메시지나 불필요한 주석들을 정리합니다. 이러한 정리 작업을 거쳐 코드를 좀 더 깔끔하게 만들 수 있고, 배포도 가능합니다. 더불어 정리 작업은 조금이지만 코드 용량을 줄이는 효과도 있습니다. 즉, 배포는 정리된 최종 결과물을 만드는 과정입니다.

과거에는 최종 결과물인 소프트웨어를 CD나 USB 등에 담아 고객에게 전달했습니다. 하드웨어 같은 장비들은 ROM이라는 저장 장치에 저장해서 배포했습니다. 요즘에는 인터넷이 발달하면서 과거의 물리적인 배포 대신 온라인으로 배포합니다. 인터넷이 연결된 서버에서 최종 결과물의 파일을 직접 내려받을 수 있습니다. 온라인 배포는 물리적 생산 비용과 유통 비용이 들지 않고, 빠르게 배포할 수 있습니다. 이렇게 배포 프로세스가 빨라지면서 코드를 안정적으로 유지하고 테스트하는 것이 더 중요해졌습니다.

따라서 개발자는 코드를 안정적인 상태로 유지하면서 쉽게 배포할 수 있는 도구들이 필요해졌습니다. 배포를 잘하려면 코드를 깔끔하게 정리할 수 있는 환경이 필요합니다. 또 사용자가 파일을 쉽게 내려받을 수 있게 해야 합니다. 이러한 이유로 깃은 코드를 배포하는 데도 많이 사용합니다.

10.2 버전

세상에 완벽한 것은 없습니다. 코드도 마찬가지입니다. 배포 당시에는 발견하지 못했는데, 이후에 잘못된 동작을 하거나 예외적인 상황이 생겨 처리하지 못하는 동작들이 있을 수 있습니다. 또는 최종 사용자가 추가 기능을 요구할 때도 있습니다. 이때는 코드를 수정해야 합니다. 즉, 코드는 개발을 완료한 후에도 계속 수정됩니다.

코드를 수정했다면 개발자 또는 최종 사용자가 이를 확인하고 구별할 수 있어야 합니다. 코드에서 이러한 차이를 구별할 수 있게 하는 것이 바로 버전(version)입니다. 버전은 프로그램을 수정하거나 개선할 때마다 코드를 구분하려고 부여된 식별자를 의미하며, 보통 숫자를 사용합니다. 숫자가 클수록 최근에 수정된 코드입니다. 버전업(version up)은 오래된 버전의 프로그램을 최신 버전의 코드로 변경하는 것을 의미합니다.

버전 번호를 부여할 때는 약간의 규칙이 있습니다. 이 규칙은 오래 전부터 업계에서 사용하던 형태로 꼭 사용할 필요는 없지만, 가능하면 따르는 것을 권장합니다.

기본적인 버전은 단일 번호 하나로 구성되어 있습니다. 단일 번호는 프로그램에서 큰 기능을 변경했을 때 바뀝니다. 첫 자리가 0으로 시작하면 아직 초기 개발 중인 제품이라는 의미입니다. 정식 버전은 1부터 시작합니다. 이를 메이저(major) 버전이라고 합니다. 메이저 번호를 변경하면 하위 버전과 호환성이 낮아질 수도 있습니다.

메이저 버전 다음으로 작은 코드의 변화는 점(.)을 사용하여 구분합니다. 보통 두 자리 또는 세 자리 형태의 숫자로 작성합니다.

- 두 자리: 1.0
- 세 자리: 2.1.4

10.3 태그

깃에서는 코드 배포를 관리하려고 태그(tag) 기능을 제공합니다. 커밋은 코드 변화를 기록한 것입니다. 그리고 각 커밋은 서로 다른 코드의 상태를 가집니다. 물론 배포를 위해 최종 정리된 커밋도 있습니다. 깃은 정리된 커밋을 배포할 수 있도록 특수한 포인터를 제공하며, 특정 커밋을 가리키는 포인터로 버전을 관리합니다. 그리고 이 포인터를 태그라고 합니다. 즉, 태그는 특정 커밋의 해시 값을 가리키는 꼬리표를 의미합니다.

최종 사용자는 개발자가 부여한 태그를 사용하여 코드 버전을 구별합니다. 또 태그 포인터로 최종 배포판의 커밋을 구별합니다. 태그는 커밋 해시 값을 기준으로 생성됩니다. 그리고 특정 커밋 해시 값을 가리키는 것뿐만 아니라, 꼬리표 이름과 정보도 포함합니다. 이러한 태그는 추가 정보를 보유하는지 여부에 따라 두 가지로 구분합니다.

- Annotated: 태그 이름 + 정보 포함
- Lightweight: 태그 이름만 포함

이 두 가지 태그의 생성 방법과 활용법은 뒤에서 실습과 함께 알아보겠습니다.

10.4 태그 목록

태그 기능을 본격적으로 학습하기에 앞서 간단히 태그 명령어를 알아봅시다. 태그를 사용하려면 tag 명령어를 사용합니다. 단독으로 실행할 수도 있고, 옵션을 추가할 수도 있습니다.

$ git tag

옵션은 -l 또는 --list 옵션을 같이 사용합니다.

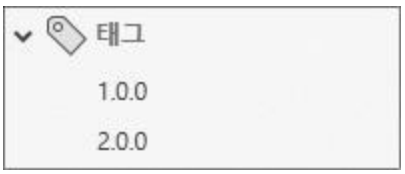
예

```
$ git tag -l
```

Note

소스트리에서는 생성된 태그 목록을 좀 더 쉽게 확인할 수 있습니다. 화면 왼쪽에 태그 탭을 표시하는데, 태그 탭을 선택해서 확장하면 생성된 모든 태그 목록을 확인할 수 있습니다.

▼ 그림 10-1 태그 목록



10.5 Annotated 태그

Annotated 태그는 깃에서 가장 일반적으로 사용하는 태그 방법입니다. Annotated는 '주석이 달린'이라는 뜻입니다.

10.5.1 태그 생성

Annotated 태그를 생성할 때는 커밋의 해시 값뿐 아니라 추가로 생성자 정보를 같이 넣을 수 있습니다. 예를 들어 이메일, 날짜, 메시지 등 정보입니다. 또 GPG 방식으로 서명도 가능합니다.

Annotated 태그를 생성하려면 tag 명령어 뒤에 -a 옵션을 사용합니다.

$ git tag -a 버전

바로 실습해 보겠습니다. version.htm 파일을 수정하고 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ code version.htm VS Code 실행
```

version.htm

```
<h1>태그 버전 실습 파일입니다.</h1>


<ul>


    <li>버전 1.0.0 실습</li>


</ul>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ git commit -am "test 1.0.0" 등록 및 커밋


[main 53028dc] test 1.0.0


 1 file changed, 4 insertions(+), 1 deletion(-)
```

10.5.2 간단한 메시지

Annotated 태그를 생성할 때는 메시지를 작성해야 합니다. 작성할 태그 정보가 간단하다면 vi 에디터를 사용하지 않고, -m 옵션으로 대체할 수 있습니다. 커밋 명령어의 -m 옵션과 유사합니다.

이어서 실습해 보겠습니다. version.htm 파일에 한 줄을 더 추가하고 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ code version.htm VS Code 실행
```

version.htm

```
<h1>태그 버전 실습 파일입니다.</h1>


<ul>


    <li>버전 1.0.0 실습</li>


    <li>버전 1.1.0 실습</li>


</ul>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ git commit -am "test 1.1.0" 등록 및 커밋


[main da8d211] test 1.1.0


 1 file changed, 1 insertion(+)
```

10.5.3 소스트리에서 태그 생성

소스트리에서도 태그를 작성할 수 있습니다. 실습을 위해 version.htm 파일을 수정하고 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ code version.htm VS Code 실행
```

version.htm

```
<h1>태그 버전 실습 파일입니다.</h1>

<ul>

    <li>버전 1.0.0 실습</li>

    <li>버전 1.1.0 실습

        <ul>

            <li>수정 작업 1.1.1</li>

        </ul>

    </li>

</ul>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git commit -am "test 1.1.1" 등록 및 커밋

[main f2691a0] test 1.1.1

 1 file changed, 5 insertions(+), 1 deletion(-)
```

10.5.4 태그는 중복해서 생성할 수 없다

깃에 등록된 태그 이름은 유일해야 합니다. 즉, 태그는 같은 이름으로 중복해서 생성할 수 없습니다. 실습으로 확인해 봅시다.

version.htm 파일을 수정하고 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ code version.htm VS Code 실행
```

version.htm

```
<h1>태그 버전 실습 파일입니다.</h1>

<ul>

    <li>버전 1.0.0 실습</li>

    <li>버전 1.1.0 실습

        <ul>

            <li>수정 작업 1.1.1</li>

            <li>수정 작업 1.1.2</li>

        </ul>

    </li>

</ul>
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git commit -am "test 1.1.2" 등록 및 커밋

[main 80f8890] test 1.1.2

 1 file changed, 1 insertion(+)
```

10.5.5 태그 삭제

태그는 특정 커밋을 가리키는 꼬리표입니다. 실수로 생성할 태그의 커밋을 잘못 지정할 수도 있습니다. 이때는 기존에 생성한 태그를 삭제해야 합니다.

태그는 tag -d 명령어로 삭제할 수 있습니다.

$ git tag -d 태그이름

태그 목록에서 삭제된 태그 이름은 이후에 다시 사용할 수 있습니다.

지금까지 진행한 실습의 로그를 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ git log --oneline 커밋 로그


80f8890 (HEAD → main) test 1.1.2


f2691a0 (tag: 1.1.1) test 1.1.1


da8d211 (tag: 1.1.0) test 1.1.0


53028dc (tag: 1.0.0) test 1.0.0


3c92e35 first
```

커밋 f2691a0에 태그 1.1.1이 지정되어 있습니다. 태그 1.1.1을 삭제해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git tag -d 1.1.1 태그 삭제

Deleted tag '1.1.1' (was b3ffc7d)
```

10.5.6 태그의 상세 정보 확인: show 명령어

Annotated 태그는 커밋의 태그 포인터와 함께 여러 정보를 포함합니다. tag 명령어는 태그의 목록만 출력할 뿐 상세 정보는 표시하지 않습니다. 태그의 상세 정보를 확인하려면 show 명령어를 사용해야 합니다.

$ git show 태그이름

다음과 같이 show 명령어를 실행하면 생성된 태그의 상세 정보를 확인할 수 있습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git show 1.0.0 태그 정보

tag 1.0.0

Tagger: hojin <infohojin@gmail.com>

Date:   Thu May 23 18:12:16 2019 +0900



this is first tag 태그 정보



commit 53028dc1486b42d23253ffd4001a758cef455372 (tag: 1.0.0)

Author: hojin <infohojin@gmail.com>

Date:   Thu May 23 18:07:18 2019 +0900

    test 1.0.0



diff --git a/version.htm b/version.htm
```

```
index 14f6513..eccf619 100644

--- a/version.htm

+++ b/version.htm

@@ -1 +1,4 @@

-<h1>태그 버전 실습 파일입니다.</h1>

\ No newline at end of file

+<h1>태그 버전 실습 파일입니다.</h1>

+<ul>

+    <li>버전 1.0.0 실습</li>

+</ul>

\ No newline at end of file
```

10.6 Lightweight 태그

Lightweight 태그는 가장 기본적인 태그입니다. Annotated 태그와 달리 버전 이름만 있습니다.

10.6.1 체크섬

Annotated 태그에는 커밋 해시 값과 부가적인 정보가 같이 있습니다. 반면 Lightweight 태그 방식은 커밋의 체크섬만 가지고 있습니다. 또 Annotated 태그에서 사용했던 -a, -m 같은 옵션은 사용할 수 없습니다.

$ git tag 태그이름

Lightweight 태그의 사용법은 간단합니다. 명령어 뒤에 태그 이름만 작성하면 됩니다. Lightweight 태그를 생성해 보겠습니다. 실습을 위해 version.htm 파일을 수정하고 저장한 후 커밋합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ code version.htm VS Code 실행
```

version.htm

```
<h1>태그 버전 실습 파일입니다.</h1>

<ul>

    <li>버전 1.0.0 실습</li>

    <li>버전 1.1.0 실습

        <ul>

            <li>수정 작업 1.1.1</li>

            <li>수정 작업 1.1.2</li>

        </ul>

    </li>

    <li>버전 2.0.0 lightweight 실습</li>

</ul>
```

10.6.2 태그의 상세 정보 확인

생성된 Lightweight 태그의 상세 정보를 확인해 보겠습니다. Annotated 태그와 동일하게 show 명령어를 사용합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git show 2.0.0 태그 정보

commit 3ba46c0a20a9812341e057818ef155f6801d0760 (HEAD → main, tag: 2.0.0) Lightweight 태그는 이 위치에 정보가 없음



Author: hojin <infohojin@gmail.com>

Date:   Thu May 23 19:14:41 2019 +0900

    test 2.0.0

diff --git a/version.htm b/version.htm

index 78de597..c9c61dc 100644

--- a/version.htm

+++ b/version.htm

@@ -7,4 +7,5 @@

             <li>수정 작업 1.1.2</li>

         </ul>
```

```
     </li>


+    <li>버전 2.0.0 lightweight 실습</li>


 </ul>


\ No newline at end of file
```

Annotated 태그와 달리 별도의 정보가 없는 것을 확인할 수 있습니다.

10.7 특정 커밋 태그

tag 명령어는 기본적으로 현재 HEAD가 가리키는 커밋을 기준으로 태그를 생성합니다. 현재 HEAD 포인터가 가리키는 커밋이 아닌 특정 커밋을 직접 지정하여 태그를 생성할 수 있습니다.

$ git tag -a 태그버전 커밋ID

특정 커밋을 지정하여 커밋을 생성할 때는 마지막 옵션으로 커밋 해시 값을 적습니다. 그러면 tag 명령어는 지정된 커밋 해시 값을 기준으로 새로운 태그를 생성합니다.

커밋을 지정해서 태그를 생성해 봅시다. 먼저 로그를 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ git log --oneline 커밋 로그


3ba46c0 (HEAD → main, tag: 2.0.0) test 2.0.0


80f8890 test 1.1.2 이곳을 지정해서 생성함


f2691a0 test 1.1.1


da8d211 (tag: 1.1.0) test 1.1.0


53028dc (tag: 1.0.0) test 1.0.0


3c92e35 first
```

10.7.1 소스트리에서 특정 커밋 지정

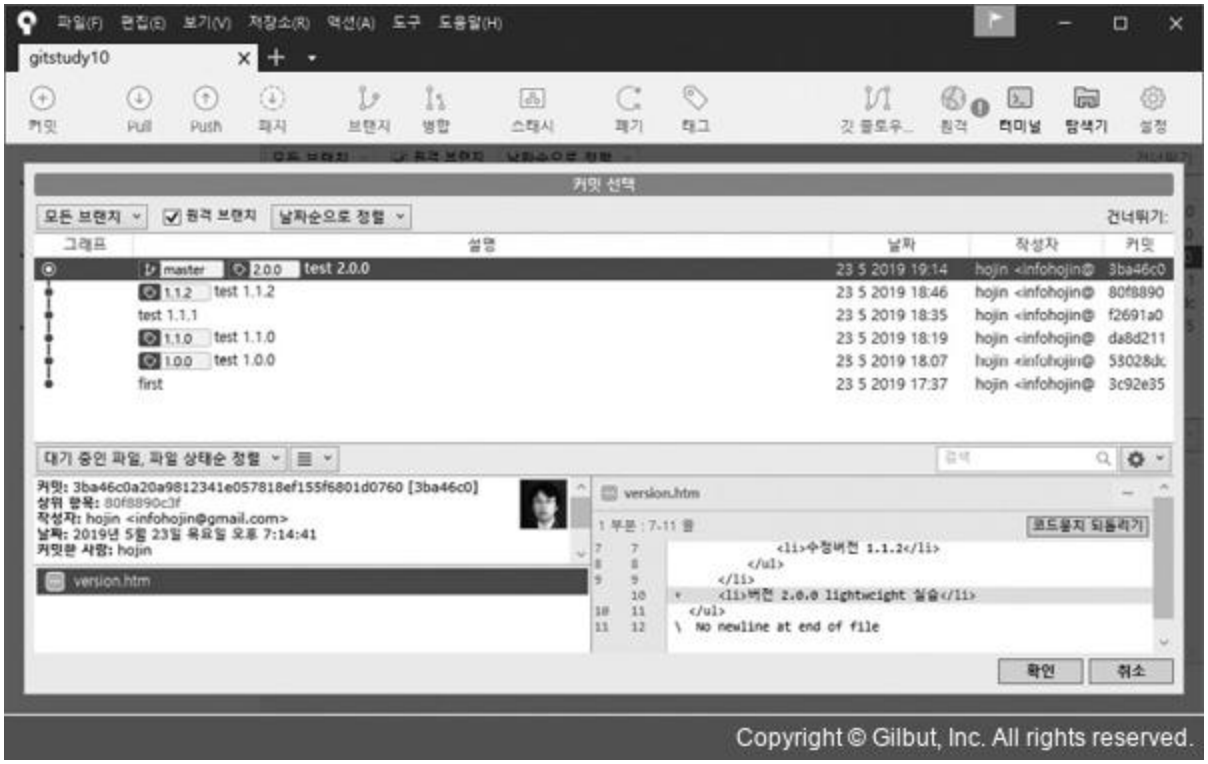
소스트리에서도 특정 커밋을 지정하여 태그를 생성할 수 있습니다. 가장 쉬운 방법은 그래프 목록에서 커밋을 선택하는 것입니다. 커밋을 클릭한 후 마우스 오른쪽 버튼을 눌러 명시적 커밋 메뉴를 선택합니다. 책에서는 test 2.0.0 커밋에서 마우스 오른쪽 버튼을 눌러 명시적 커밋 메뉴를 선택하고, 태그 이름에는 2.0.0을 입력합니다.

▼ 그림 10-10 특정 커밋(test 2.0.0)의 태그 생성



마지막으로 태그 추가를 누르면 다음과 같이 2.0.0 태그가 생성됩니다.

▼ 그림 10-11 명시적 태그 추가



10.8 태그를 사용한 체크아웃

태그는 특정 커밋을 가리키는 포인터입니다. 따라서 태그를 사용하여 특정 커밋으로 체크아웃할 수 있습니다. 체크아웃할 때 브랜치 이름 대신 태그 이름을 입력하면 됩니다.

$ git checkout 태그이름

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git checkout 1.0.0 태그로 브랜치 이동

Note: checking out '1.0.0'.



You are in 'detached HEAD' state. You can look around, make experimental

changes and commit them, and you can discard any commits you make in this

state without impacting any branches by performing another checkout.



If you want to create a new branch to retain commits you create, you may

do so (now or later) by using -b with the checkout command again. Example:



  git checkout -b <new-branch-name>



HEAD is now at 53028dc test 1.0.0
```

infoh@DESKTOP MINGW64 /e/gitstudy10 ((1.0.0)) 브랜치 이름이 태그 이름으로 출력됨

10.8.1 태그 브랜치

태그를 사용하여 체크아웃할 때는 추가 커밋을 작성할 수 없습니다. 추가 커밋 작업이 필요하다면 태그를 기반으로 새 브랜치를 생성합니다. 태그로 생성된 브랜치는 동일한 시작 커밋을 가지며, 새 브랜치에서 원하는 작업을 추가로 할 수 있습니다.

$ git checkout -b 브랜치이름 태그이름

태그로 브랜치를 생성할 때는 커밋의 해시 값 대신 태그 이름을 사용합니다. 그럼 실습해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 ((1.0.0))


$ git checkout -b fix 1.0.0 태그 기준으로 브랜치 생성


Switched to a new branch 'fix'




infoh@DESKTOP MINGW64 /e/gitstudy10 (fix)
```

태그 이름으로 새 브랜치를 생성하고, 체크아웃까지 동시에 할 수 있습니다.

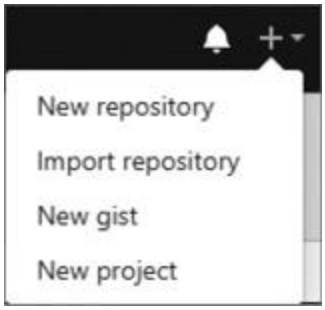
10.9 태그 공유

지금까지는 로컬 저장소에서 태그를 생성하는 방법을 알아보았습니다. 로컬 저장소에서 생성한 태그를 원격 저장소로 공유할 수 있습니다. 태그는 불특정 다수에게 최종 소스의 커밋 정보를 알리려고 공유하는 것입니다. 즉, 태그를 공유하는 것은 최종 코드를 배포하는 것과 같습니다. 태그를 사용하여 사용자는 최종 코드를 내려받을 수 있습니다.

10.9.1 원격 저장소 생성

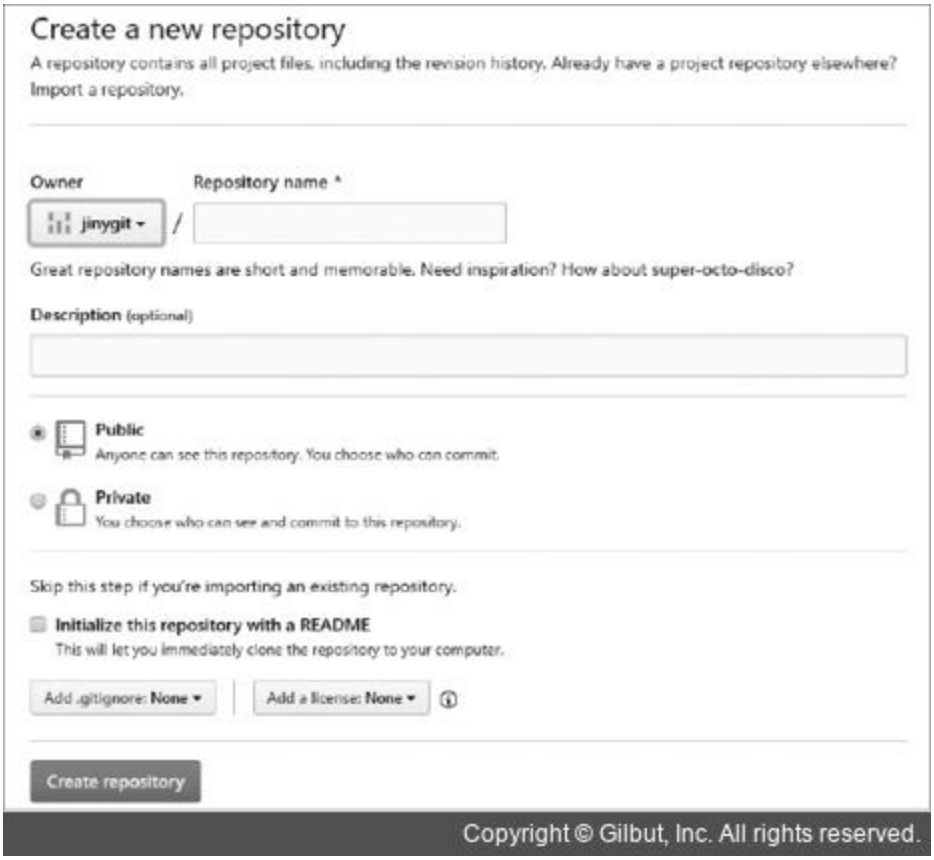
태그 정보를 공유하는 데는 원격 저장소가 필요합니다. 깃허브 같은 호스팅 서비스를 이용하여 공개 저장소를 생성합니다. 책에서는 5장에서 생성한 깃허브 저장소를 사용하겠습니다. 먼저 깃허브에 로그인한 후 New 버튼 또는 + > New repository를 클릭합니다.

▼ 그림 10-12 깃허브에서 새 저장소 메뉴 선택



다음 화면에서 원하는 저장소 이름을 입력하고 Create repository를 누릅니다. 필자는 gitstudy10으로 정했는데, 로컬 저장소 이름과 동일하지 않아도 됩니다.

▼ 그림 10-13 원하는 이름으로 새 저장소 생성



10.9.2 태그 동기화

생성된 원격 저장소로 로컬 저장소의 태그 정보를 공유할 수 있습니다. 태그 정보는 원격 저장소로 자동 전송되지 않으므로 직접 동기화 명령을 수행해야 합니다.

로컬 저장소의 커밋을 원격 저장소로 전송할 때는 push 명령어를 사용합니다. 원격 저장소로 로컬 저장소의 브랜치는 다음과 같이 전송할 수 있습니다.

$ git push 브랜치이름

직접 실습해 봅시다. 로컬 저장소의 main 브랜치를 원격 저장소로 전송해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git push origin main 커밋 전송

Enumerating objects: 18, done.

Counting objects: 100% (18/18), done.

Delta compression using up to 8 threads

Compressing objects: 100% (11/11), done.

Writing objects: 100% (18/18), 1.44 KiB | 113.00 KiB/s, done.

Total 18 (delta 4), reused 0 (delta 0)

remote: Resolving deltas: 100% (4/4), done.

To https://github.com/jinygit/gitstudy10.git

 * [new branch]      main → main
```

10.9.3 전체 태그 동기화

이전에는 태그 1개를 지정하여 원격 저장소로 공유했습니다. 로컬 저장소에는 태그가 여러 개 있습니다. 이 태그를 하나씩 전송하는 것은 불편합니다. 한꺼번에 전송할 수도 있습니다. 실습으로 알아봅시다.

지금까지 실습한 로컬 저장소의 태그를 다시 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git tag 태그 목록

1.0.0

1.1.0

1.1.2

2.0.0
```

태그가 많군요. push 명령어의 --tags 옵션을 사용하면 로컬 저장소의 모든 태그를 한꺼번에 원격 저장소로 보낼 수 있습니다.

$ git push origin --tags

로컬 저장소의 모든 태그를 원격 서버로 전송해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)

$ git push origin --tags 모든 태그 전송

Enumerating objects: 1, done.

Counting objects: 100% (1/1), done.

Writing objects: 100% (1/1), 156 bytes | 39.00 KiB/s, done.

Total 1 (delta 0), reused 0 (delta 0)

To https://github.com/jinygit/gitstudy10.git

 * [new tag]         1.1.0 → 1.1.0

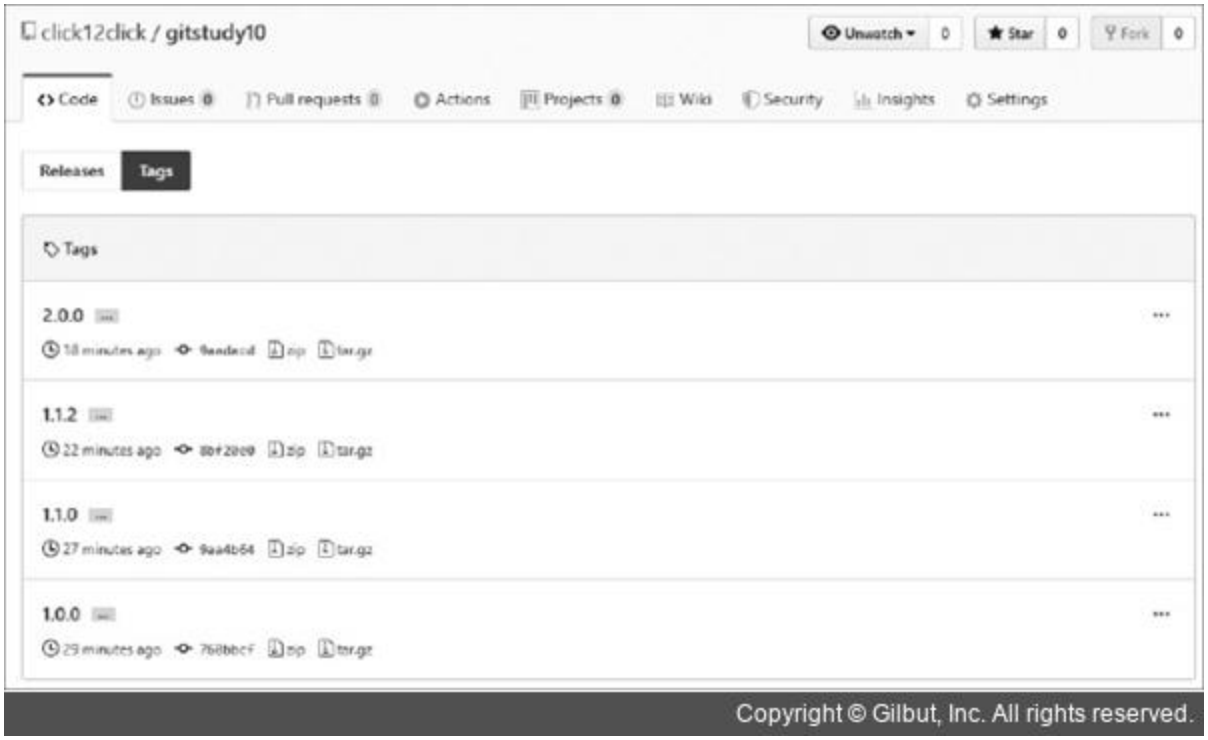
 * [new tag]         1.1.2 → 1.1.2

 * [new tag]         2.0.0 → 2.0.0
```

10.9.4 원격 저장소의 태그 수정과 삭제

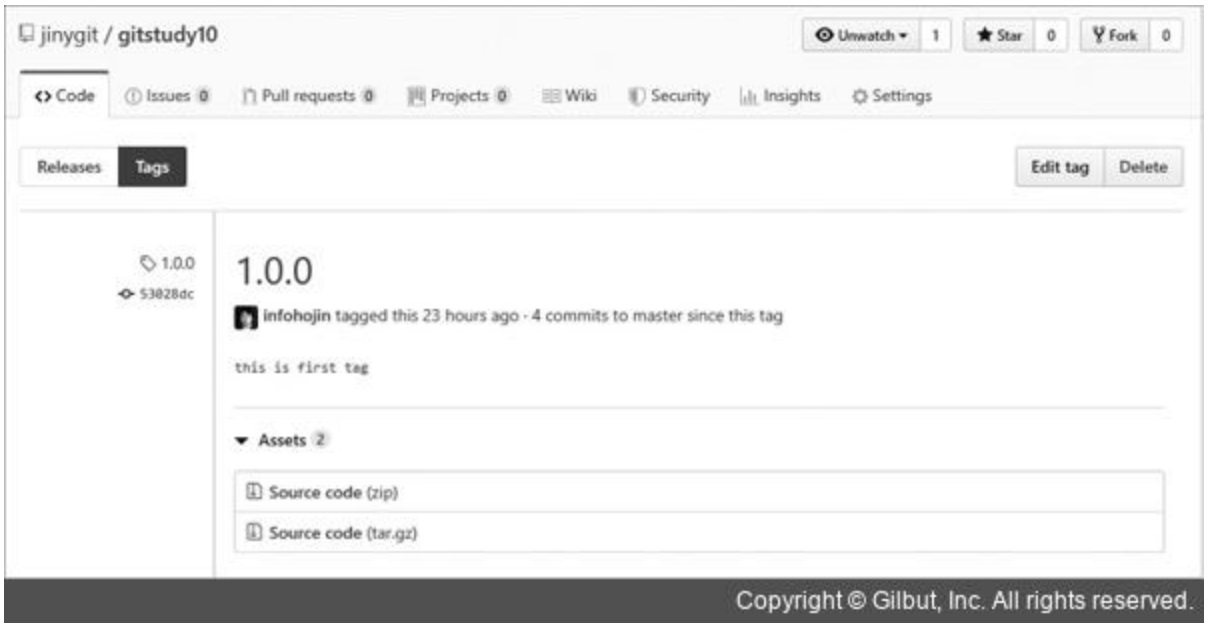
원격 저장소에 등록된 태그를 깃허브에서 직접 수정할 수 있습니다. 깃허브에서 Tags를 누른 후 수정할 태그를 클릭합니다.

▼ 그림 10-20 깃허브에서 수정할 태그 선택



선택한 화면에서 오른쪽 위의 Edit tag를 누릅니다.

▼ 그림 10-21 태그 수정



10.9.5 원격 저장소에 로컬과 다른 이름으로 태그 전송

push 명령어는 로컬 저장소의 정보를 원격 저장소로 전송합니다. 이때 push 명령어를 사용하여 로컬 저장소의 브랜치를 원격 저장소의 브랜치로 전송할 경우, 다른 이름으로 전송할 수 있었습니다.

예

```
$ git push origin main:main1
```

이 예는 로컬 저장소의 main 브랜치를 origin 서버(원격 저장소)의 main1 브랜치로 등록하는 방법입니다. 이 원리를 응용하여 태그도 동일한 이름이 아닌 다른 이름으로 전송할 수 있습니다. 다음과 같이 콜론(:)을 사용하여 로컬 저장소의 태그 이름과 원격 저장소의 태그 이름을 지정합니다.

$ git push origin 태그이름 : 원격저장소의태그이름

로컬 저장소의 1.1.2 태그를 원격 저장소에는 3.0.0 태그로 등록해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy10 (main)


$ git push origin 1.1.2:3.0.0 다른 이름의 태그 전송

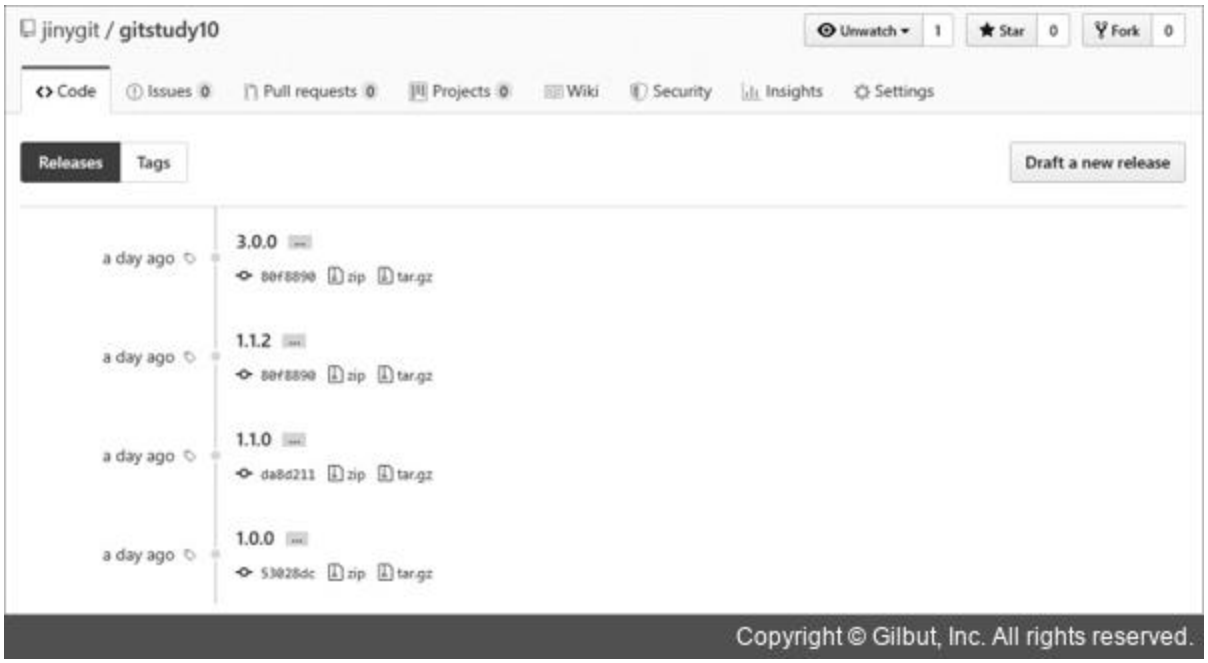

Total 0 (delta 0), reused 0 (delta 0)


To https://github.com/jinygit/gitstudy10.git


 * [new tag]         1.1.2 → 3.0.0
```

깃허브 저장소를 확인합니다. 변경된 3.0.0 태그가 등록되었네요. 주의 깊게 살펴보면 1.1.2 태그와 3.0.0 태그가 모두 동일한 80f8890 해시 값을 가리킵니다.

▼ 그림 10-23 로컬 저장소와 다른 이름의 태그 전송


jinygit / gitstudy10
Unwatch 1
Star 0
Fork 0
Code
Issues 0
Pull requests 0
Projects 0
Wiki
Security
Insights
Settings
Releases
Tags
Draft a new release
a day ago
3.0.0
80f8890 zip tar.gz
a day ago
1.1.2
80f8890 zip tar.gz
a day ago
1.1.0
da8d211 zip tar.gz
a day ago
1.0.0
53028dc zip tar.gz
Copyright © Gilbut, Inc. All rights reserved.

10.10 정리

태그는 완성된 코드를 배포하고 관리하는 데 매우 유용합니다. 또 최종 사용자는 태그를 확인하여 어떤 커밋이 완성된 코드인지 쉽게 확인할 수 있습니다.

태그 버전은 패키지들이 배포하고 업데이트하는 기준이기 때문에 PHP의 컴포저나 Node.js의 패키지 등을 개발할 때도 유용하게 사용할 수 있습니다.

실제 현장에서는 코드를 개발하는 과정도 중요하지만, 태그를 사용하여 버전을 유지 관리하는 것도 매우 중요합니다. 아무리 잘 만든 제품이라도 배포와 태그를 잘 관리하지 않는다면 최종 사용자들은 혼돈에 빠지게 됩니다.